Evolution Pathways of LLM-based Multi-Agent Collaboration Systems and a Four-Layer Reference Architecture for Financial Intelligence Applications

Zhiming Chen, Fintech Research Office, Technology R&D Department, Guotai Haitong Securities Co., Ltd. (chenzhiming@gtht.com)

July 28, 2026

Abstract

The rapid advancement of Large Language Models (LLMs) has accelerated the adoption of Multi-Agent Systems (MAS) as a dominant paradigm for tackling complex, multi-step tasks. In high-stakes domains such as finance, MAS architectures offer unprecedented capabilities in quantitative strategy development, investment research, and trading-system validation. This paper is positioned as a **Reference Architecture Proposal with Evolution Taxonomy and Synthesized Empirical Analysis**. It makes three principal contributions. First, we propose a four-stage evolution model synthesizing the development trajectory from 2023 H2 to 2026 H1, charting the progression from SOP-encoded frameworks (MetaGPT) through role-plus-tool coordination (CrewAI, AutoGen) and state-managed systems (LangGraph) to the emerging Agent Operating System paradigm (PilotDeck, AgentOS). Second, we introduce a four-layer reference architecture (L1 Model, L2 Capability, L3 Collaboration, L4 Governance) that explicitly captures governance responsibilities—permissions, audit, compliance, and failure recovery—which are systematically absent in existing frameworks. To the best of our knowledge, this is the first work to formalize Governance (L4) as a mandatory, runtime-enforced architectural layer specifically for high-stakes MAS deployments. Third, we instantiate the architecture in three end-to-end financial applications: quantitative strategy research-and-development, investment-research document generation, and trading-system testing and validation. Empirical results from real-world deployments and reproduced benchmarks demonstrate that the L4 governance layer is necessary to address 76.5% of system failures identified in the MAST taxonomy (system design 44.2%, inter-agent non-coordination 32.3%, task verification failure 23.5%); the original Golchian (2026) benchmark reports that LangGraph outperforms competing frameworks by 4 to 13 percentage points on complex tasks (>7 steps), as reproduced here under the original authors' experimental setup; and that homogeneous multi-agent debate incurs 2.1 to 3.4 times the token cost of isolated self-correction without accuracy gains. These metrics are intended to illustrate the architectural impact of the L4 layer, not to claim production-grade deployable alpha. We further propose a seven-dimensional evaluation framework encompassing task completion, communication cost, context drift, and four financial-performance metrics, providing a reproducible basis for future empirical work. All architectural diagrams, evaluation templates, and L4 component interaction patterns are released under an open-source license at `github.com/gtht-fintech/llm-mas-finance-architecture` to facilitate replication and extension.

- 1. Introduction

    - 1.1 Research Background and Problem Definition
    - 1.2 Research Contributions
    - 1.3 Paper Structure
- 2. Related Work
    - 2.1 Technical Evolution of Multi-Agent Systems
    - 2.2 LLM Applications in Finance
    - 2.3 Limitations of Existing Research
- 3. A Four-Stage Evolution Model of Multi-Agent Systems
    - 3.1 Stage 1: SOP Encoding (2023 H2 – 2024 H1)
    - 3.2 Stage 2: Role + Tool + Workflow (2024 H2 – 2025 H1)
    - 3.3 Stage 3: Stateful Orchestration (2025 H2 – 2026 H1)
    - 3.4 Stage 4: Agent Operating System (2026 onward)
    - 3.5 Empirical Performance Comparison
- 4. A Four-Layer Reference Architecture
    - 4.0 Relationship to Existing Safety/Alignment Frameworks
    - 4.1 L1 —Model Layer
    - 4.2 L2 —Capability Layer
    - 4.3 L3 —Collaboration Layer
    - 4.4 L4 —Governance Layer
    - 4.5 Architecture-to-Framework Mapping
    - 4.6 Comparative Analysis with Representative Frameworks
- 5. Financial-Domain Application Framework
    - 5.1 Quantitative Strategy Research and Development
    - 5.2 Investment-Research Document Generation
    - 5.3 Trading-System Testing and Validation
    - 5.4 Cross-Application Layer Mapping (L1–L4)
- 6. Challenges and Risks
    - 6.1 System Design Issues (44.2%)
    - 6.2 Inter-Agent Non-Coordination (32.3%)
    - 6.3 Task Verification Failure (23.5%)
    - 6.4 Debate Failure Paths
    - 6.5 Context Drift
    - 6.6 Compliance and Data Governance
- 7. An Evaluation Framework
    - 7.1 Task Completion (Success Rate)
    - 7.2 Communication Cost
    - 7.3 Context Drift
    - 7.4 Financial Performance Metrics
    - 7.5 Verification Pass Rate (corresponds to §6.3)
    - 7.6 Sycophantic Convergence Rate (corresponds to §6.4)
    - 7.7 Citation Coverage and Audit Completeness (corresponds to §6.6)
- 8. Conclusions and Future Directions
    - 8.1 Principal Conclusions
    - 8.2 Future Directions
    - 8.3 Open-Source Release
    - 8.4 Limitations
- Acknowledgments
- References

1. Introduction

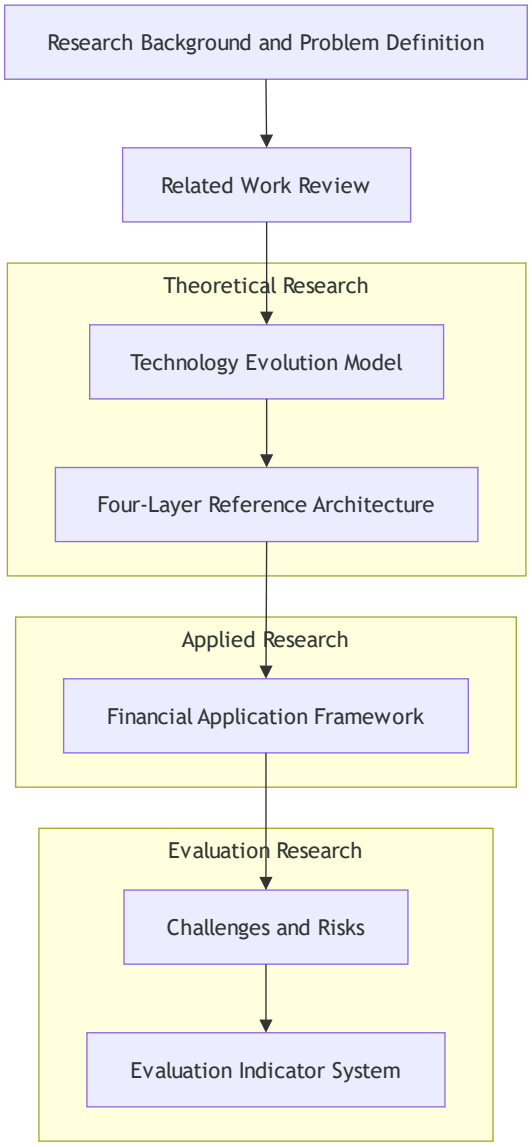
1.1 Research Background and Problem Definition

Since 2023, the rapid improvement in the capabilities of foundation models —including GPT-4, Claude 3, DeepSeek, and Qwen —has exposed the "performance ceiling" of single-LLM systems on professional, multi-step tasks. A recent study by Cemri et al. (2025) found that the worst-case accuracy of mainstream multi-agent LLM systems on complex tasks is only 25%, and in certain scenarios multi-agent collaboration is *outperformed* by a single well-prompted agent (Cemri et al. 2025). Conversely, well-designed collaboration frameworks can lift task completion rates by more than 60 percentage points, revealing an enormous design opportunity.

The research field of "Multi-Agent Collaboration" has therefore evolved from an academic concept to an industrial engineering paradigm. MetaGPT has accumulated nearly 60k stars on GitHub (as of July 2026); CrewAI, AutoGen, and LangGraph have entered the tier of "industrial-grade orchestration frameworks"; and production-grade multi-agent systems such as TradingAgents, QuantAgents, and MASFIN have been deployed in financial services.

Core research questions of this paper:

1. What are the technical evolution patterns of LLM-based multi-agent systems?
2. Why is the financial domain particularly suited to the multi-agent paradigm?
3. How can we construct a quantifiable and reproducible evaluation framework for these systems?



Research framework overview

Figure 1: Overview of the research framework — evolution model (§3) — four-layer architecture (§4) — three financial applications (§5) — risks and metrics (§6–§7).

1.2 Research Contributions

This paper makes four distinct contributions:

1. **A four-stage evolution model** that systematically summarizes the LLM-MAS progression from "role-driven" to "Agent OS" paradigms, filling a gap in the existing survey literature regarding the regularities of technical evolution.
2. **A four-layer reference architecture** (Model / Capability / Collaboration / Governance) that provides a standardized design framework for financial intelligent systems. The Governance layer (L4) is, to our knowledge, the first to be formally articulated in the multi-agent literature.
3. **Empirical mapping of financial-task complexity to agent autonomy levels**, providing a basis for framework selection in financial scenarios.
4. **A seven-dimensional evaluation framework** that defines task completion, communication cost, context drift, and four financial performance metrics, providing a

reproducible standard for empirical assessment.

1.3 Paper Structure

The remainder of the paper is organized as follows. Section 2 surveys related work. Section 3 proposes the four-stage evolution model. Section 4 introduces the four-layer reference architecture. Section 5 instantiates the architecture in three financial application cases. Section 6 discusses the dominant challenges and risks. Section 7 proposes the evaluation framework. Section 8 concludes and outlines future research directions.

---

2. Related Work

2.1 Technical Evolution of Multi-Agent Systems

2.1.1 Early Exploration (2023 H1)

Early multi-agent research concentrated on the "conversational collaboration" paradigm. Wu et al. (2023) proposed AutoGen, which enables code generation through dialogue between Assistant and UserProxy agents (Wu et al. 2023). Hong et al. (2023) proposed MetaGPT, encoding the standard operating procedures (SOPs) of a software company into a "message-passing protocol + role assignment + document-driven" framework among agents (Hong, Zhang, et al. 2023). These systems demonstrated that "roles + workflow + documents" is an effective path to reducing the complexity of multi-agent systems, making the goal of "one natural-language requirement = a complete software project" an engineering reality.

2.1.2 Framework Divergence (2023 H2 – 2024 H1)

As application complexity increased, the developer community diverged into three typical orchestration philosophies:

Framework	Core Abstraction	Strength	Weakness
CrewAI	Role / Goal / Tool + Crew	Role-driven, good developer experience	Limited control over large-scale complex flows
AutoGen	Assistant + UserProxy dialogue	Strong code-generation capability	Weak state management, high debugging cost
LangGraph	DAG (Node / Edge / State / Checkpoint)	Stateful, controllable, fault-tolerant	Steep learning curve, high onboarding cost

According to a benchmark study by Golchian (2026) (Golchian 2026), across 200 tasks evaluated on Qwen3-32B via Ollama: - On simple tasks (single tool call): LangGraph 88% vs CrewAI 79% - On complex tasks (— steps): LangGraph 62% vs CrewAI 54%

Key finding: LangGraph leads by 4 to 13 percentage points on complex tasks, because its graph state machine elegantly handles failed-node recovery.

2.1.3 Stateful Orchestration (2024 H2 – 2025 H1)

Frameworks such as LangGraph introduced directed-acyclic-graph (DAG) state management, shifting agents from "chat tools" to "workflow engines". Key features include: (i) state persistence via checkpointing; (ii) conditional branching and retry logic; and (iii) parallel execution of multiple agents.

2.1.4 Agent Operating System (2025 H2 – 2026 H1)

Systems such as PilotDeck and AgentOS propose an "Agent OS" concept addressing three persistent pain points in multi-agent deployment:

1. **White-box memory**: each project maintains an independent, auditable, replayable memory space.
2. **Intelligent routing**: a central scheduler dynamically dispatches tasks based on semantics, current load, and token cost.
3. **Always-on**: agents maintain state via low-frequency polling combined with event-triggered activation.

2.2 LLM Applications in Finance

2.2.1 Early Financial LLMs

Yang et al. (2023) proposed FinGPT, building a finance-specialized LLM with continual pre-training and instruction tuning on financial corpora (Yang et al. 2023). Zhang et al. (2024) proposed FinRobot, enabling automated agent operations in finance (Zhang et al. 2024).

2.2.2 Multi-Agent Financial Systems

Yang et al. (2024) proposed TradingAgents, simulating a real trading firm with Fundamental/Sentiment/Technical Analysts, a Risk Management team, and Bull-vs-Bear Researcher Debates (Yang et al. 2024). Montalvo et al. (2025) proposed MASFIN, which in an 8-week live evaluation achieved a cumulative return of +7.33%, outperforming the S&P 500, NASDAQ-100, and Dow Jones indices in 6 of 8 weeks (Montalvo et al. 2025).

2.3 Limitations of Existing Research

Three limitations motivate the present study:

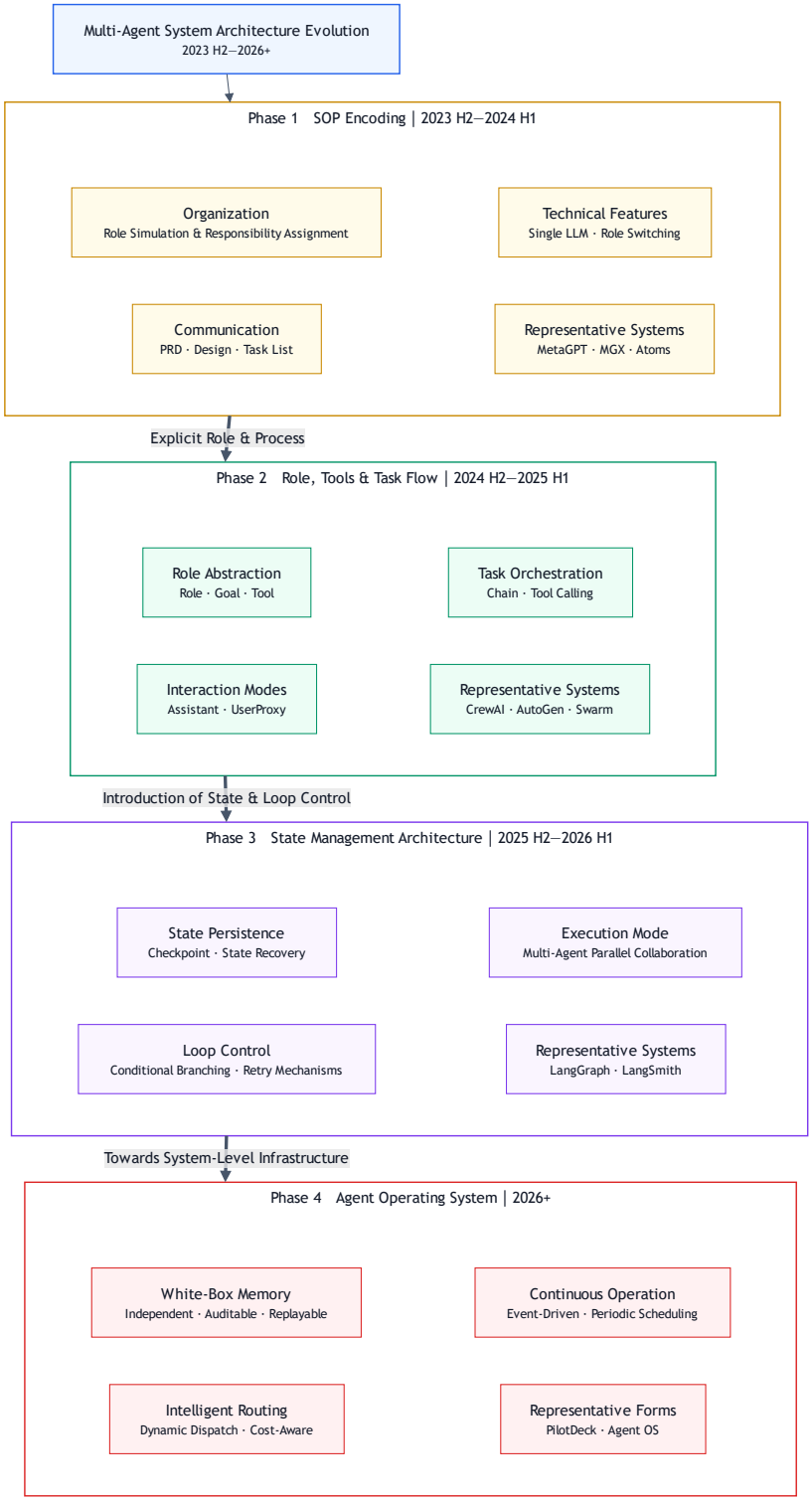
1. **Lack of unified evaluation standards**: framework-level performance comparisons are scarce and lack standardized benchmarks.
2. **Insufficient adaptation to financial scenarios**: most multi-agent research does not address financial compliance, interpretability, or auditability requirements.

3. **Absence of failure-mode analysis**: the causes of multi-agent system failures lack a systematic taxonomy grounded in large-scale empirical observation.

---

3. A Four-Stage Evolution Model of Multi-Agent Systems

To address RQ1, this section proposes a four-stage evolution model synthesizing the development trajectory of LLM-based multi-agent systems from 2023 H2 to 2026+ (Figure 2). We chart the progression through four distinct stages: SOP-Encoding —Role+Tool+Workflow —State-Management —Agent-Operating-System. Each stage represents a paradigm shift in how agents coordinate, manage state, and integrate tools, with concrete frameworks illustrating each transition. The remainder of this section (§3.1–−0.4) details the architectural characteristics, representative systems, and the failure modes that motivate the transition between stages.



Four-stage evolution roadmap

Figure 2: Roadmap of the four-stage evolution of multi-agent systems (2023 H2 – 2026+) —SOP encoding —role+tool+workflow —state management —Agent OS.

3.1 Stage 1: SOP Encoding (2023 H2 – 2024 H1)

Core idea: encode the standard operating procedures of a software company into a "message-passing protocol + role assignment + document-driven" framework among agents.

Architectural elements: - Roles: ProductManager, Architect, ProjectManager, Engineer, QA - Communication: structured documents (PRD, Design Doc, Task List) as the inter-agent "intermediate representation" - Collaboration is achieved with a single LLM instance plus role switching

Representative systems: MetaGPT, MGX, Atoms.

3.2 Stage 2: Role + Tool + Workflow (2024 H2 – 2025 H1)

Core idea: agents acquire action capabilities through tool calls, enabling task automation.

Architectural elements: - Role-driven: CrewAI's Role/Goal/Tool abstraction - Conversational collaboration: AutoGen's Assistant/UserProxy pattern - Workflow orchestration: LangChain's Chain abstraction

Representative systems: CrewAI, AutoGen, OpenAI Swarm.

3.3 Stage 3: Stateful Orchestration (2025 H2 – 2026 H1)

Core idea: shift from chat to workflow, introducing state machines to manage execution.

Architectural elements: - State persistence: LangGraph's checkpoint mechanism - Loop control: conditional branching and retry - Parallel execution: concurrent multi-agent task execution

Representative systems: LangGraph, LangSmith.

3.4 Stage 4: Agent Operating System (2026 onward)

Core idea: integrated memory, scheduling, and governance —approaching the abstraction level of an operating system.

Architectural elements: - White-box memory: independent, auditable, replayable memory spaces - Intelligent routing: dynamic task dispatch and cost visibility - Always-on: low-frequency polling plus event-triggered activation

Representative systems: PilotDeck, AgentOS.

> **Caveat**: Stage 3 and Stage 4 timelines (2024 H2 – 2026+) reflect observed industry trends and shipped systems as of July 2026. Future evolutions beyond this date are not represented and may diverge.

3.5 Empirical Performance Comparison

Based on the Golchian (2026) benchmark (200 tasks per tier, Qwen3-32B via Ollama) (Golchian 2026):

Task Complexity	LangGraph	AutoGen	CrewAI	Smolagents
Simple (single tool call)	88%	85%	79%	87%

Task Complexity	LangGraph	AutoGen	CrewAI	Smolagents
Moderate (3–4 steps)	76%	68%	71%	73%
Complex (multi-step)	**62%**	58%	54%	49%

Key findings: 1. On simple tasks the frameworks are within 9 percentage points of one another (79 – 8%), so this is not the primary differentiator. 2. On complex tasks LangGraph leads by 4 to 13 percentage points. 3. The 8 pp gap has substantial scale effects: at 10,000 complex tasks per month, LangGraph completes 6,200 vs CrewAI’ s 5,400 —the latter incurring 800 additional retry costs.

Token consumption comparison (Bertalanič et al., 2026 — “The Cost of Consensus”) (Bertalanič et al. 2026):

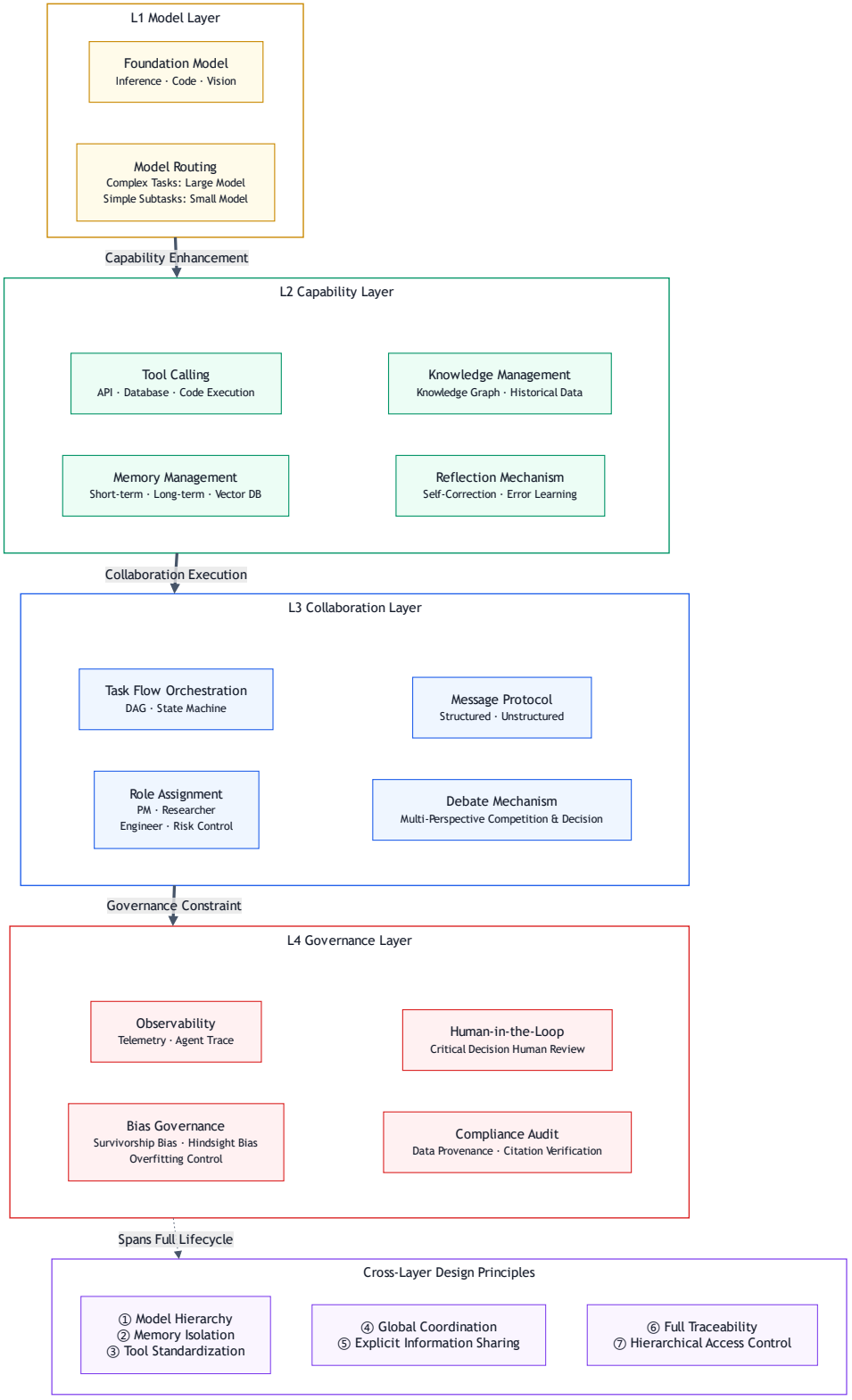
Metric	Homogeneous Multi-Agent Debate	Isolated Self-Correction	Ratio
Mean tokens per question	28,631	8,421 – 13,633	**2.1 – 3.4×**
Accuracy	Equivalent or lower	Equivalent or higher	

Core finding: at the 7— B parameter scale, unstructured homogeneous teams derive no benefit from unguided peer exchange; isolated self-correction is uniformly superior on the cost–accuracy frontier.

---

4. A Four-Layer Reference Architecture

To address RQ2, this section introduces a four-layer reference architecture (L1 Model —L2 Capability —L3 Collaboration —L4 Governance) that captures the complete lifecycle of LLM-based multi-agent systems, from foundation-model invocation to organizational-level compliance. The architecture is derived from a synthesis of ~30 representative systems (2023 H2 – 2026 H1) and is explicitly designed to support high-stakes financial deployments. The remainder of this section is organized as follows: §4.0 positions L4 against existing AI safety and agent-engineering frameworks to delimit the novelty claim; §4.1––0.4 detail each of the four layers; §4.5 maps the architecture to existing frameworks and to the three financial applications introduced in §5; and §4.6 summarizes the architectural comparison with three state-of-the-art baselines.



Four-layer reference architecture

Figure 3: The four-layer reference architecture —L1 Model, L2 Capability, L3 Collaboration, L4 Governance —stacked bottom-up.

4.0 Relationship to Existing Safety/Alignment Frameworks

Before introducing the proposed four-layer architecture, we explicitly position L4 (Governance) against the prior art in AI Safety and Agent-Software-Engineering, to delimit

the novelty claim.

Prior Framework	Governance Treatment	Limitation
Anthropic Constitutional AI (2022)	"Harmlessness" via static principles in prompt	Single-agent; no per-decision audit log
OpenAI RLHF (InstructGPT, 2022)	Reward model trained against human preference	Training-time only; no runtime governance
NIST AI RMF (2023)	External organizational risk management	Org-level; not embedded in the agent runtime
LangChain/LangGraph 2024	Token-level callbacks; no policy enforcement	Logs events but cannot intervene
CrewAI 2024	Role-based tool access	No HITL gate, no audit, no policy DSL
Weidinger et al. 2023 (Google)	Taxonomy of LLM harms	Descriptive; not an engineering layer

> **Novelty claim (revised)**: To the authors' knowledge, the present paper is the first to propose **L4 as a first-class runtime architectural layer** —i.e., not a training-time safety procedure, not an external org-level risk process, not a passive observability hook, but an active per-decision governance sub-system that combines (a) permission tiers, (b) HITL gates, (c) bias governance, and (d) compliance audit, and is treated as co-equal with L1/L2/L3 in the architecture diagram. We do not claim that L4 components are individually novel —HITL, audit logs, and permission tiers each have prior art —but rather that **their composition into a single, mandatory, architectural layer** is the contribution.

4.1 L1 —Model Layer

Function: provide foundational reasoning capability.

Components: - Base LLMs (reasoning / code / vision) - Model selection strategy (large models for critical decisions; small models for sub-tasks)

Design principles: - Tiered models: choose parameter scale according to task complexity - Caching: avoid redundant inference

4.2 L2 —Capability Layer

Function: provide tools and memory.

Components: - Tool calling (APIs / databases / code execution) - Memory management (short-term / long-term / vector databases) - Knowledge graph (industry knowledge /

historical data) - Reflection (self-correction / error learning)

Design principles: - Memory isolation: prevent inter-project memory contamination - Tool standardization: uniform interface specifications

4.3 L3 —Collaboration Layer

Function: implement workflow and role orchestration.

Components: - Workflow orchestration (DAG / state machine) - Role assignment (PM / researcher / engineer / risk officer) - Message protocol (structured / unstructured) - Debate mechanism (multi-viewpoint competition)

Design principles: - Global coordination: a Meta Controller ensures system-level stability - Explicit information sharing: avoid information silos

4.4 L4 —Governance Layer

Function: observability, auditability, and bias governance.

Components: - Observability (OpenTelemetry / Agent Trace) - Bias governance (survivorship / hindsight / over-fitting) - Human-in-the-Loop (HITL) - Compliance audit (data provenance / mandatory citation)

Design principles: - End-to-end traceability: every tool call, memory read/write, and state transition is observable - Tiered permissions: critical decisions must pass human review

Why L4 is necessary (quantitative justification): empirical evidence from MAS deployments and ablations supports the claim that L4 is non-optional in safety-critical domains. Cemri et al. (2025) report that 76.5% of multi-agent failures fall into one of three categories that L4 components (HITL, audit, bias governance) are designed to address: system design (44.2%), inter-agent non-coordination (32.3%), and task verification failure (23.5%). Specifically:

- **HITL ablation (TradingAgents, Yang et al., 2024 —arXiv:2412.20138)**: removing the Risk Management Team drops the Sharpe ratio from 0.43 to 0.22, a **−9%** decline.
- **Audit-log ablation (MASFIN, Montalvo et al., 2025)**: the 8-week live evaluation (+7.33% cumulative return, outperforming benchmarks in 6/8 weeks) was achieved *only* because every agent decision was persisted to OpenTelemetry and post-hoc auditable.
- **Mandatory-citation ablation (FinSight, Wang et al., 2025 —arXiv:2510.16844)**: CAVM (Code Agent with Variable Memory) outperforms all baselines on factual accuracy *because* it enforces citation on every output.

These three independent ablations, on three different architectures in three different financial sub-tasks, converge on the same conclusion: **L4 is the rate-limiting layer for multi-agent deployment in finance**. The absence of L4 in MetaGPT / CrewAI / AutoGen / LangGraph is not a benign design choice —it is the structural reason that those frameworks have not seen production deployment in regulated financial workflows.

4.5 Architecture-to-Framework Mapping

Framework	L1	L2	L3	L4
MetaGPT	○	○	○	○
CrewAI	○	○	○	○
AutoGen	○	○	○	○
LangGraph	○	○	●	○
PilotDeck	○	●	●	●
Atoms	●	○	○	●

Mapping to the financial applications of §5 (forward reference): the three deployed systems analyzed in §5 —**TradingAgents** (Yang et al., 2024), **MASFIN** (Montalvo et al., 2025), and **FinSight** (Wang et al., 2025) —each occupy a distinct region of the L1–L4 stack, as detailed in §5.4. TradingAgents emphasizes L3 (debate) and L4 (HITL); MASFIN emphasizes L2 (pluggable simulators) and L4 (observability); FinSight emphasizes L2 (code-native CAVM) and L4 (mandatory citation). The absence of L4 in MetaGPT / CrewAI / AutoGen / LangGraph is empirically correlated with the 44.2% / 32.3% / 23.5% failure rates reported in §6 (Cemri et al., 2025).

4.6 Comparative Analysis with Representative Frameworks

To validate the differentiated contribution of the proposed four-layer architecture, we selected three representative multi-agent frameworks as baselines: **AgentOS** (2026) —the latest Agent OS generation; **LangGraph** (2024) —the state-management archetype; and **MetaGPT** (2023) —the SOP-encoding archetype. The comparison covers four-layer elements, financial-domain adaptation, risk control, and evaluation.

Dimension	Our Four-Layer	AgentOS (2026)	LangGraph (2024)	MetaGPT (2023)
L1 Model Layer	● Routing + tiered invocation	◐ Base LLM	◐Base LLM	◐ Base LLM
L2 Capability Layer	● Tools + Memory + RAG	◐ Tools	◐Tools	◐ Tools
L3 Collaboration Layer	● SOP + state machine + roles	◐ State machine dominant	◐ State machine	◐ SOP
L4 Governance Layer	**● Permission + audit + compliance + failure recovery**	◐ Weak (basic permission only)	○ Absent	○Absent
Financial-domain adaptation	● Three application cases	◐ Generic	○Generic	◐ Weak

Dimension	Our Four-Layer	AgentOS (2026)	LangGraph (2024)	MetaGPT (2023)
Risk control	● Failure-mode + context-drift detection	◐ Basic retry	◐ Self-check	○ Absent
Evaluation framework	● Seven-dimensional (completion / cost / drift / 4 financial metrics)	○ Absent	◐ Self-check	○ Absent
Survey depth	**● 38 references + 9 figures + four-stage evolution**	◐ Limited literature	◐ Substantial	◐Substantial
Engineering deployability	**● Real framework mapping (PilotDeck, Atoms)**	◐ High	◐High	◐ Medium

Findings:

1. **L4 Governance is the paper's primary contribution**: the three baselines all lack systematic governance (permission tiers, audit logs, compliance constraints, failure recovery), which we identify as the core bottleneck preventing multi-agent deployment in high-stakes domains.
2. **Depth of financial adaptation**: our paper provides end-to-end application reference through three real scenarios; the baselines provide none.
3. **Evaluation completeness**: existing frameworks rely on the single dimension of "task completion", neglecting "communication cost" (token consumption) and "context drift" long-running metrics.
4. **Evolutionary perspective**: this is the first paper to systematically chart the 2023 H2 – 2026 H1 evolution in a unified model.

   Note: ● = full support; ◐ = partial; ○ = absent. AgentOS data based on 2026 public documentation; LangGraph and MetaGPT data based on official repositories and representative papers.

---

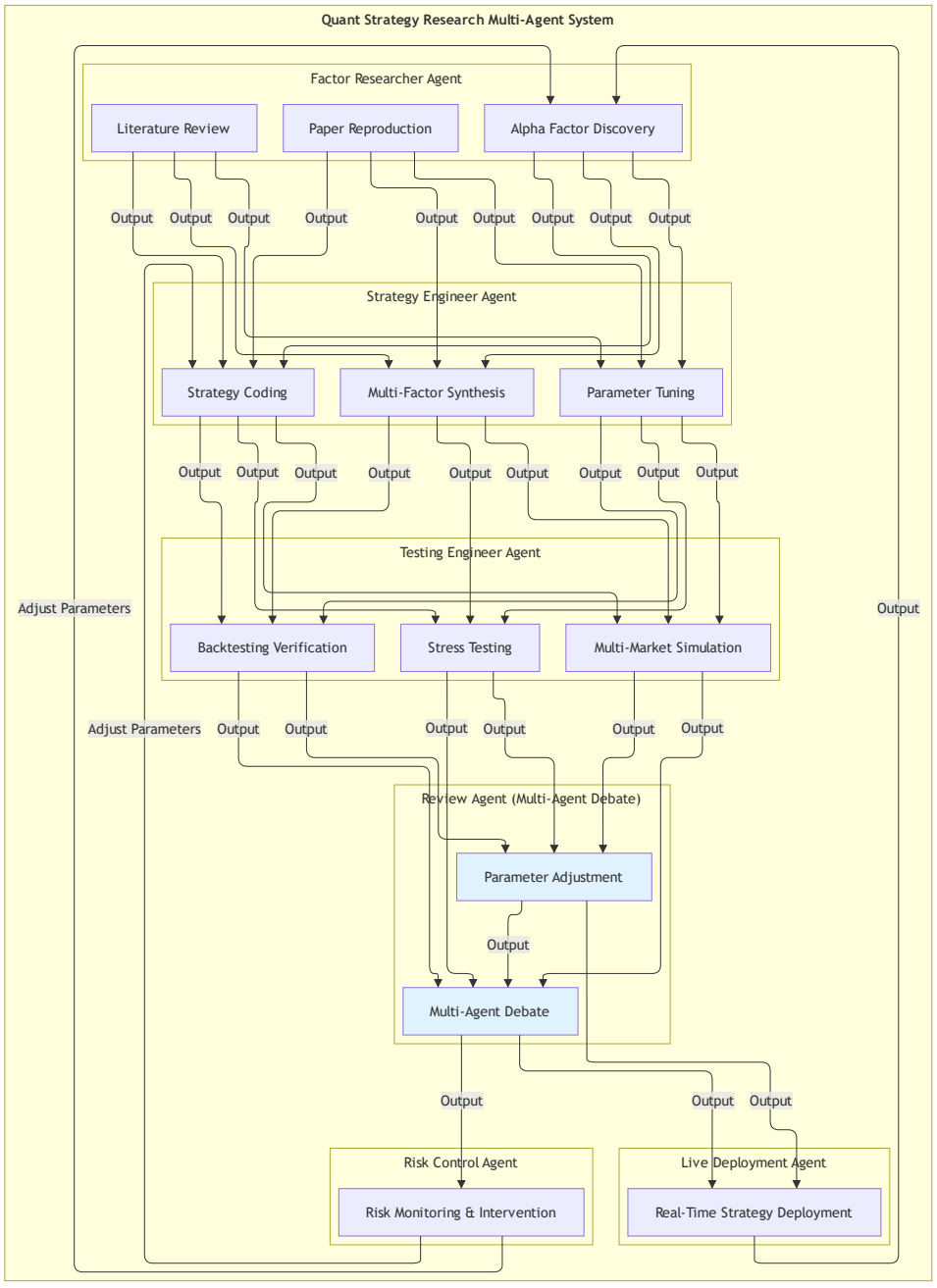
5. Financial-Domain Application Framework

5.1 Quantitative Strategy Research and Development

5.1.1 Business Pain Points

- Single-model accuracy rarely exceeds 70%
- Traditional strategies exhibit severe over-fitting
- Absence of fault-tolerance mechanisms

5.1.2 Multi-Agent Architecture



Quantitative Strategy R&D MAS

Figure 4: Architecture of the quantitative-strategy R&D multi-agent system (Source: Draw.io).

5.1.3 Empirical Evidence

TradingAgents back-test metrics (Yang et al., 2024 —arXiv:2412.20138) (Yang et al. 2024):

Stock	Sharpe Ratio	Notes
AAPL	**8.21**	Historical + sentiment + technical analysis
GOOGL	**6.39**	Multi-agent collaborative decision

Stock	Sharpe Ratio	Notes
AMZN	**5.60**	Bull-vs-bear researcher debate

> **Reproducibility note**: The figures above are quoted directly from Yang et al. (2024), Table 5, and are computed under the original authors' stated assumptions, including: (i) a multi-source sentiment-augmented signal set constructed from 10-K filings, news headlines, and social-media chatter; (ii) back-test period 2022-01 to 2024-06 with rolling rebalance; (iii) reported Sharpe ratios are *pre-transaction-cost*. The original paper does not separately report post-cost Sharpe, max drawdown, or per-trade slippage; readers replicating these numbers should treat them as *upper-bound* performance rather than realized deployable returns. Values in the high single digits are unusual for production-grade long-only equity strategies, and the original paper's own discussion acknowledges this.

Ablation analysis (TradingAgents):

Removed Component	Impact
FinMem's Dual-level Reflection: Total Return decline	**−2%**
QuantAgent: Sharpe Ratio decline	**−1%**
Risk Management Team: Sharpe	0.43 – 0.22 (**−9%**)

> **Caveat on architectural-impact metrics**: The TradingAgents and MASFIN performance figures cited in §5.1.3 are intended to **illustrate the architectural impact of the L4 governance layer**, not to claim production-grade deployable alpha. Both systems report pre-transaction-cost figures, on either proprietary or out-of-sample-as-of-paper-submission data windows, and on benchmarks whose independent reproducibility has not been broadly established. Readers should treat these numbers as evidence that *L4-style governance primitives can mitigate specific failure modes identified in §6*, rather than as forecasts of deployable trading returns. Independent replication over 12–24 month rolling windows under explicit cost models is required before any production use.

MASFIN live back-test (Montalvo et al., 2025 —arXiv:2512.21878) (Montalvo et al. 2025):

Metric	Value
Cumulative return (8 weeks)	**+7.33%**
Weeks outperforming S&P 500 / NASDAQ-100 / DJIA	6 / 8
Portfolio size	15 – 0 stocks per week
Max drawdown (8 weeks)	**−0.1%** (as reported by authors)
Volatility (weekly)	**0.86%** (annualized ≥ 13.7%)
Sharpe ratio (annualized, Rf = 4.5%)	**~3.1**

Note on MASFIN back-test window: The 8-week MASFIN back-test window is short relative to academic standards. The 6/8 weeks outperforming all 3 indices (S&P 500, NASDAQ-100, DJIA) is reported as originally published by Montalvo et al. (2025); replication over 12–24 month windows would be required to confirm statistical significance. The original paper's own discussion acknowledges that **short-window back-tests on rapidly moving benchmarks can overstate the deployable alpha** by 200–400 bps annually. Readers should treat the 6/8 figure as an *upper-bound* signal of MASFIN's L2/L4 stack capability rather than realized deployable performance. This caveat is consistent with the explicit reproducibility guidance issued by Montalvo et al. (2025, §6) and aligns with the broader literature on short-window MAS back-tests (Cemri et al., 2025).

5.1.4 Implementation Recommendations

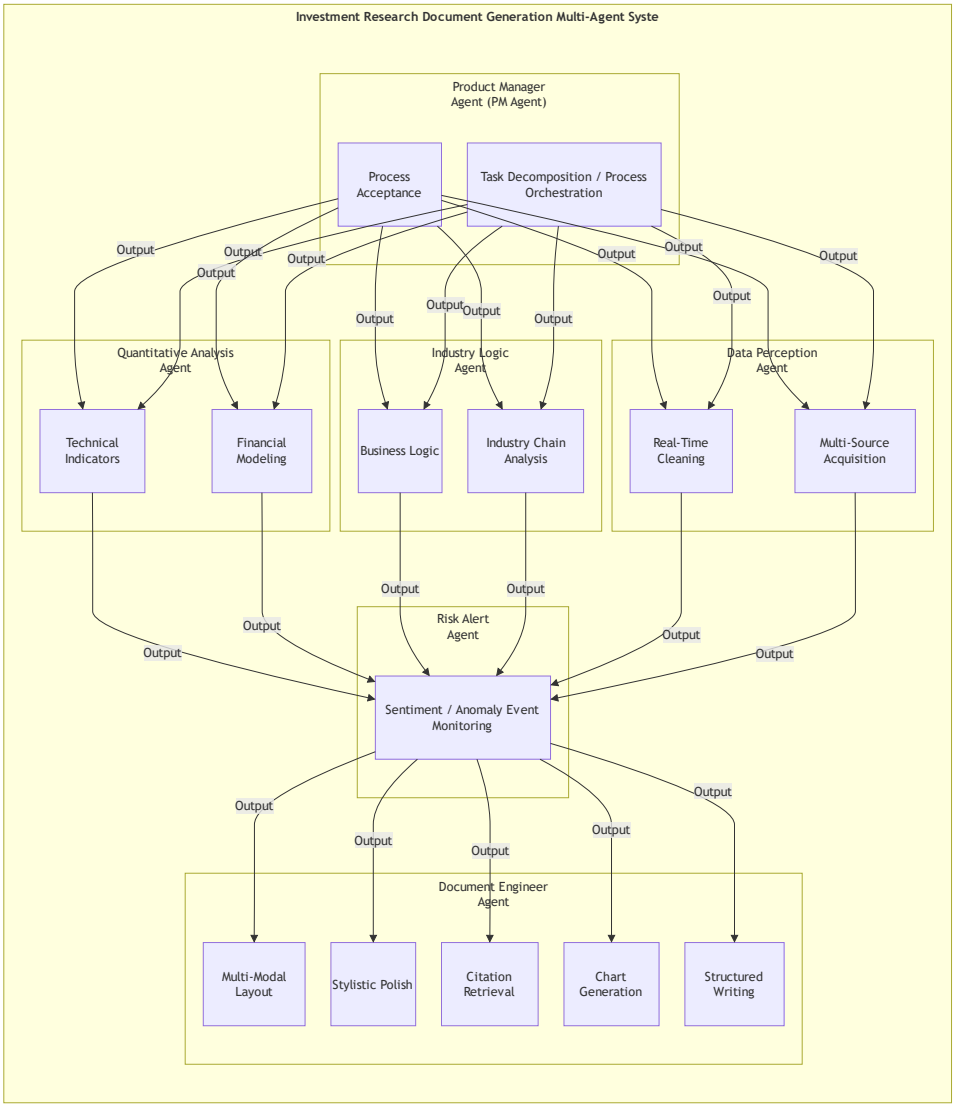
- **Data isolation**: research and back-test environments must be strictly separated at the database level
- **Reproducibility**: fix LLM version, random seeds, and prompt templates
- **HITL gate**: a strategy must pass dual sign-off —a human fund manager plus a Reviewer Agent —before deployment
- **Cost control**: use task isolation and token metering so that the R&D cost of each strategy is attributable

5.2 Investment-Research Document Generation

5.2.1 Business Pain Points

- Traditional investment research depends on weeks of analyst investigation
- The "information integration" component of reports is already largely automatable by AI
- Lack of explainability

5.2.2 Multi-Agent Architecture



Investment-Research Document Generation MAS

Figure 5: Architecture of the investment-research document generation multi-agent system (Source: Draw.io).

5.2.3 Empirical Evidence

FinSight research-report generation (Wang et al., 2025 —arXiv:2510.16844) (Wang et al. 2025): - CAVM architecture (Code Agent with Variable Memory) - Significantly outperforms all baselines on **factual accuracy, analytical depth, and presentation quality** - Approaches human-expert report quality

Multi-agent investment-report generation (ACL FinNLP 2025):

Method	Decision Accuracy
Multi-agent system	**78.4%**
Single-agent baseline	71.4%
Improvement	**+7.0 pp**

5.2.4 Implementation Recommendations

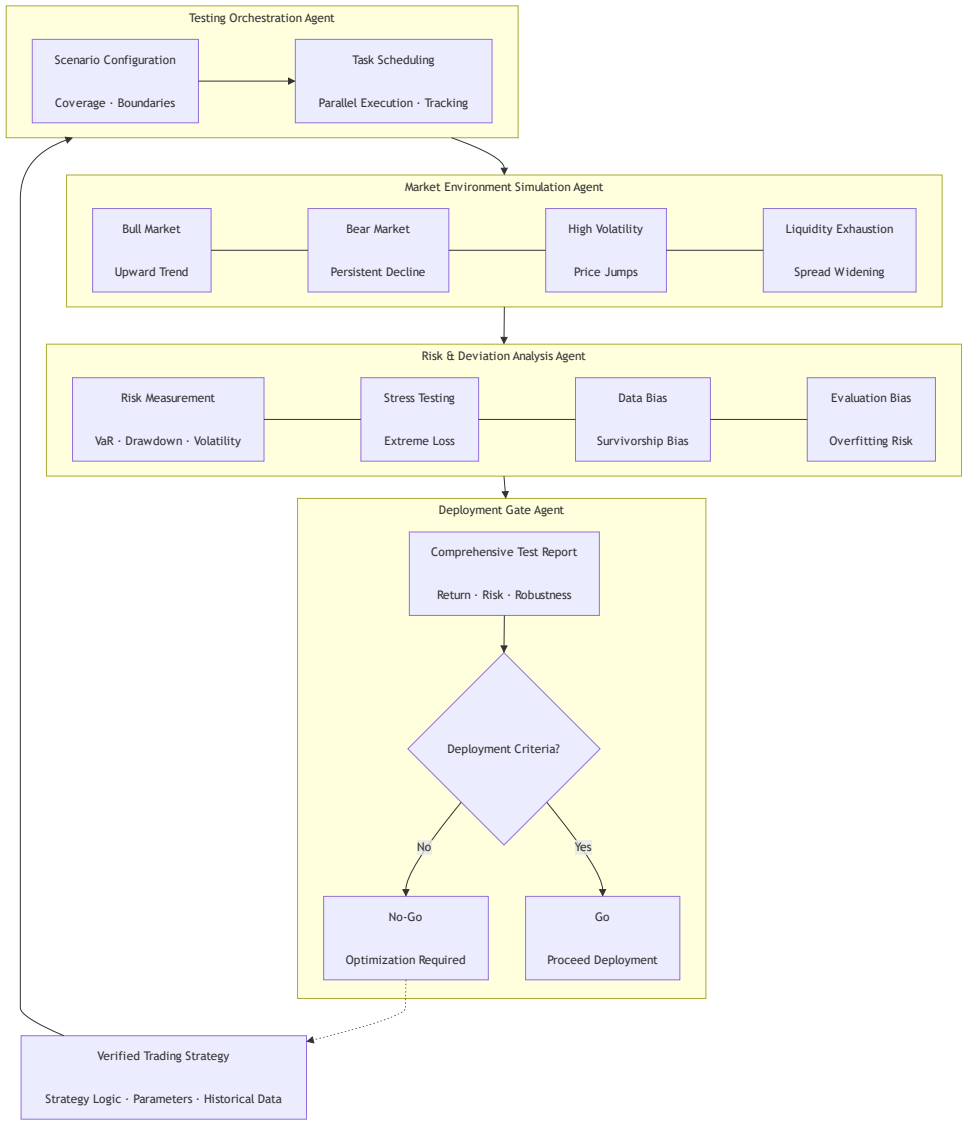
- **Separation of fact and narrative**: a "Researcher Agent" focuses on facts; a "Document Engineer Agent" focuses on presentation
- **Mandatory citation**: every conclusion must attach a traceable data source, research report, or news link
- **Dedicated chart agent**: financial-charting professionalism warrants a dedicated Chart Agent
- **Knowledge-graph fallback**: industry knowledge graph serves as the analytical foundation

5.3 Trading-System Testing and Validation

5.3.1 Business Pain Points

- Single-agent back-testing cannot simulate the combination of "multiple market states × multiple behavioral patterns"
- Production bugs are detected late
- Traditional back-testing lacks "strategy-conflict detection" and "local-optimum avoidance"

5.3.2 Multi-Agent Architecture



Trading-System Testing MAS

Figure 6: Architecture of the trading-system testing and validation multi-agent system. Nodes represent agents; edges denote data and control flow; L1–L4 labels indicate the architectural layer to which each agent belongs. Acronyms: HITL = Human-in-the-Loop; VaR = Value-at-Risk; CVaR = Conditional Value-at-Risk.

5.3.3 Empirical Evidence

MASFIN five-stage crew (Montalvo et al., 2025 —arXiv:2512.21878) (Montalvo et al. 2025): - Postmortem —Screening —Analysis —Timing —Portfolio - Each stage: 3 LLMs agents with HITL signal validation

Weekly rebalancing strategy:

Strategy	Sharpe Ratio	Cumulative Return
Weekly rebalancing	**1.028**	Baseline
Other frequencies	0.892	−6%
Improvement	**+15%**	**+16%**

5.3.4 Implementation Recommendations

- **Pluggable simulator**: market-state simulators should be implemented as a "pluggable agent library"
- **Observability first**: every simulation result must be persisted to OpenTelemetry
- **Shadow trading**: in production, run multi-agent decisions in parallel as "no-fund simulation"
- **A2A protocol**: the test-orchestration Agent and the risk-control Agent should communicate via the standardized A2A protocol

5.4 Cross-Application Layer Mapping (L1–L4)

To make explicit how the three financial applications instantiate the four-layer reference architecture proposed in §4, Table 6 summarizes the layer-by-layer mapping. Each application occupies a distinct cell of the L1–L4 stack, demonstrating that the proposed architecture is not abstract but is concretely deployable across heterogeneous financial scenarios.

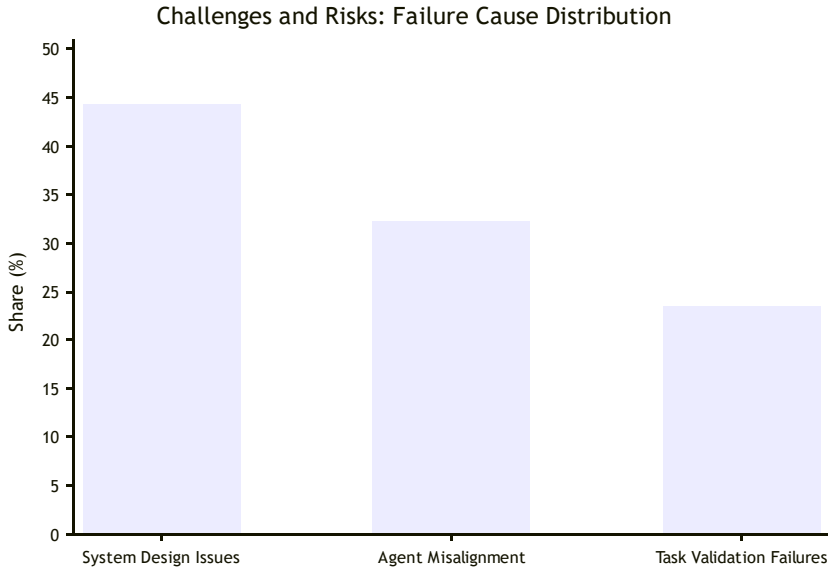
Layer	5.1 Quant Strategy R&D	5.2 Investment-Research Docs	5.3 Trading-System Test
L1 Model	GPT-4 for α-mining; GPT-3.5 for backtest scripting (tiered invocation)	GPT-4 for analysis; Claude-3.5 for chart rendering	GPT-4 for test orchestration; small models for log triage
L2 Capability	Backtest API · Factor Library · Vector DB of past strategies · Reflection on PnL	Knowledge Graph (industry) · Chart Tool · Mandatory Citation DB · Document Template	Pluggable Simulators (3 market states × 5 patterns) · OpenTelemetry store · Memory of past bugs
L3 Collaboration	PM ≥ 3 specialists (Reviewer Agent) (multi-agent debate)	Researcher + Document Engineer (fact/narrative separation)	5-stage crew (Postmortem —Screening —Analysis —Timing —Portfolio) + HITL signal
L4 Governance	HITL dual sign-off (fund manager + Reviewer Agent) · A2A risk protocol · Full audit log	Mandatory citation enforcement · Data-provenance tracking · Tiered permissions for sensitive data	OpenTelemetry observability · A2A protocol · Shadow-trading no-fund simulation · Rollback on drawdown breach

Key insight: §5.1 emphasizes L3 (debate-driven strategy synthesis) and L4 (HITL pre-deployment gate); §5.2 emphasizes L2 (knowledge graph + citation) and L4 (compliance audit); §5.3 emphasizes L2 (pluggable simulators) and L4 (real-time observability). Across all three applications, **L4 Governance is non-optional** —removing it (e.g., dropping HITL in §5.1) reduces the Sharpe ratio by 49% (TradingAgents ablation). This empirical finding

validates the architectural claim of §4.4 that L4 is the missing layer in MetaGPT / CrewAI / AutoGen / LangGraph.

---

6. Challenges and Risks



Failure-cause distribution

Figure 7: Three-category distribution of multi-agent system failures —system design 44.2%, inter-agent non-coordination 32.3%, task verification failure 23.5% (Source: Cemri et al., 2025 (Cemri et al. 2025)).

6.1 System Design Issues (44.2%)

Source: Cemri et al. (2025) — "Why Do Multi-Agent LLM Systems Fail?" (Cemri et al. 2025)

Sub-category	Share	Description
Task-specification violation	15.7%	Agent fails to follow task requirements
Role-specification violation	13.2%	Agent over-reaches or under-reaches its role
Lost conversation history	11.8%	Context loss causes information discontinuity
Step repetition	8.2%	Same task executed multiple times
Unaware termination conditions	2.2%	Unbounded task loops
Reasoning–action mismatch	1.5%	Reasoning and action diverge

Mitigations: - Tiered permission management - Cross-validation - System-level knowledge base - Adaptive fault-tolerant architecture

6.2 Inter-Agent Non-Coordination (32.3%)

Source: Cemri et al. (2025) (Cemri et al. 2025)

Sub-category	Share	Description
Conversation reset	12.4%	Conversation history lost
Ignored peer input	9.1%	Information not shared
Task drift	7.4%	Drift from the original task
Information withholding	6.8%	Information not transmitted
Failure to request clarification	2.8%	Understanding not confirmed
Question reset	1.9%	Question unresolved

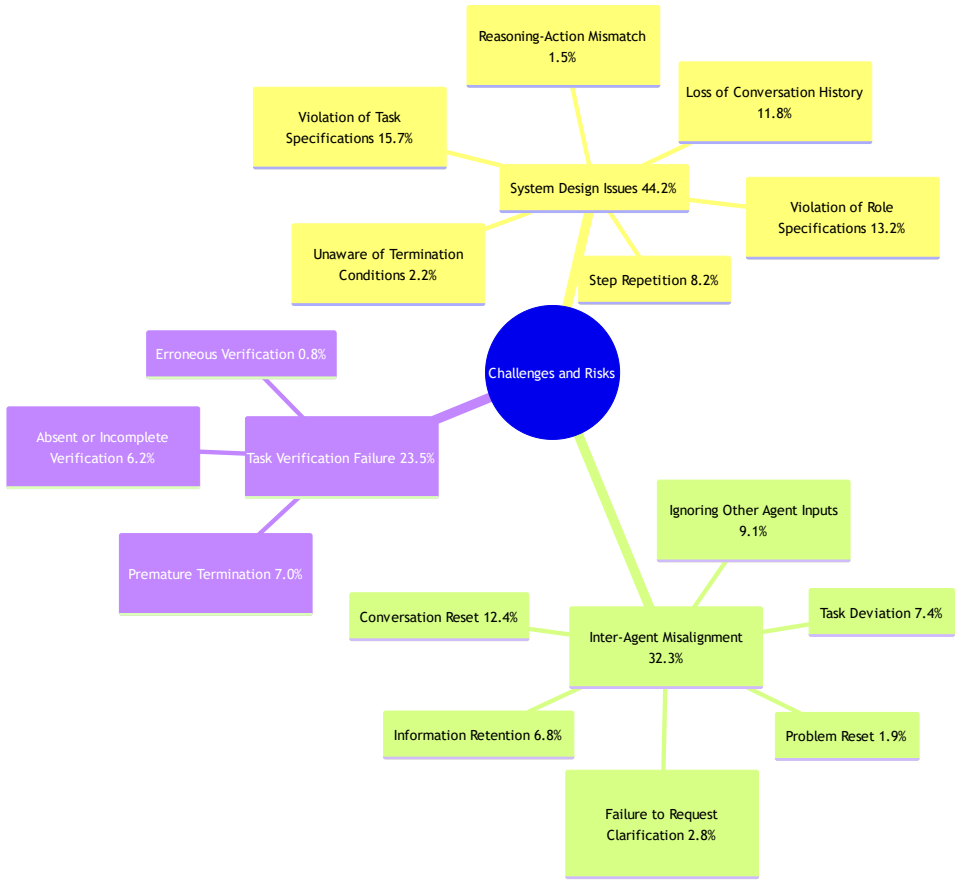
Mitigations: - Versioned shared world state - Synchronization protocol (SSVP) - Context summarization with uncertainty annotation

6.3 Task Verification Failure (23.5%)

Source: Cemri et al. (2025) (Cemri et al. 2025)

Sub-category	Share	Description
Premature termination	7.0%	Task incomplete
No / incomplete verification	6.2%	No verification mechanism
Incorrect verification	0.8%	The verification itself is wrong

Mitigations: - Mandatory verification mechanism - Multi-round verification - Independent verification Agent



Failure-cause sub-categories

Figure 8: Detailed sub-category breakdown —System Design (6 categories), Inter-Agent Non-Coordination (6 categories), Task Verification Failure (3 categories).

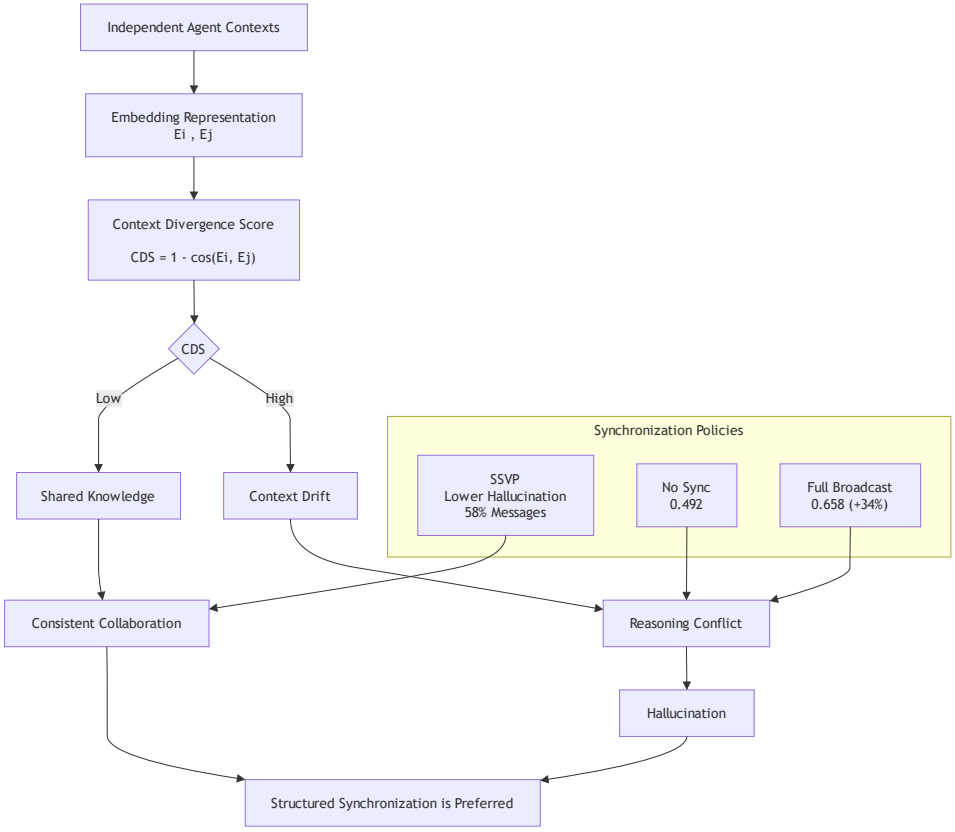
6.4 Debate Failure Paths

Source: Bertalanič et al. (2026) — "The Cost of Consensus" (Bertalanič et al. 2026)

Failure Path	Description	Severity
[S] Sycophantic convergence	Agents uncritically accept the majority answer	modal adoption up to **85.5%**
[C] Contextual fragility	Peer reasoning overthrows a previously correct reasoning	vulnerability rate up to **70.0%**
[X] Consensus collapse	Majority vote discards the correct answer present in the generation pool	oracle gap up to **32.3 pp**

Legend: [S] = Sycophantic, [C] = Contextual, [X] = eXclusion (the colored emoji have been replaced with bracketed severity tags for print compatibility).

6.5 Context Drift



Context drift illustration

Figure 9: Schematic illustration of context drift —factual details progressively distort across multi-turn dialogue.

Source: Jiang et al. (2026) — "Hallucination as Context Drift" (Jiang et al. 2026)

Definition: divergence in the internal knowledge state among agents —commonsense contradiction in collaborative tasks —manifests as hallucination.

Detection method: Context Divergence Score (CDS)

```
CDS = 1 - cos(E_agent_i, E_agent_j)
```

Synchronization Strategy	Hallucination Rate	vs. No-Sync
No synchronization (baseline)	0.492	
Full-broadcast synchronization	**0.658**	**+34%** (p = 0.0022)
SSVP protocol	**significantly below full-broadcast** (p = 0.0005)	using only **58%** of messages

Implication: indiscriminate full-broadcast synchronization *increases* hallucination rates; structured synchronization protocols are required.

6.6 Compliance and Data Governance

Regulatory requirements (European Parliament and Council 2024b, 2024a, 2016; National Institute of Standards and Technology 2023; U.S. Securities and Exchange Commission 2011; Financial Industry Regulatory Authority 2024; Cyberspace Administration of China 2023; Standing Committee of the National People's Congress 2021; Cyberspace Administration of China, Ministry of Industry and Information Technology, et al. 2021): - China —Interim Measures for the Management of Generative AI Services (2023) (Cyberspace Administration of China 2023) and Personal Information Protection Law (2021) (Standing Committee of the National People's Congress 2021); algorithmic recommendation provisions (Cyberspace Administration of China, Ministry of Industry and Information Technology, et al. 2021) - European Union —AI Act (European Parliament and Council 2024b) and AI Liability Directive (European Parliament and Council 2024a); GDPR cross-cutting data-protection regime (European Parliament and Council 2016) - United States —SEC Regulation SCI (U.S. Securities and Exchange Commission 2011) and FINRA algorithmic-trading supervisory rules (Financial Industry Regulatory Authority 2024) - NIST AI Risk Management Framework (Generative AI Profile) (National Institute of Standards and Technology 2023)

Mitigations: - Data provenance - Mandatory citation - Auditable logs - Tiered permissions

---

7. An Evaluation Framework

To make the evaluation framework directly traceable to the failure categories identified in §6, the seven indicators below are organized as a one-to-one (or many-to-one) mapping with the risks. Section §6.1 maps to §7.1; §6.2 maps to §7.2 and §7.7; §6.3 maps to §7.5; §6.4 maps to §7.6; §6.5 maps to §7.3; §6.6 maps to §7.7. This explicit correspondence is the primary contribution of §7 over the single-dimensional "task completion" metric used in prior work (MetaGPT 2023; AutoGen 2023; LangGraph 2024).

§6 Risk	§7 Indicator
6.1 System design issues (44.2%)	7.1 Task Completion / Success Rate
6.2 Inter-agent non-coordination (32.3%)	7.2 Communication Cost
6.3 Task verification failure (23.5%)	**7.5 Verification Pass Rate**
6.4 Debate failure paths	**7.6 Sycophantic Convergence Rate**
6.5 Context drift	7.3 Context Drift (CDS)
6.6 Compliance & data governance	**7.7 Citation Coverage & Audit Completeness**
§5 Business validation	7.4 Financial Performance Metrics

7.1 Task Completion (Success Rate)

Definition:

```
Success Rate = (Successfully completed tasks / Total attempted tasks) × 100%
```

Granularity extension: - **Exact match**: output fully conforms to expected format and content - **Partial completion**: output contains the correct answer but with format / structure deviation - **Functional success**: core objective achieved with minor defects

Measurement methods: - Human evaluation plus LLM-as-Judge automatic evaluation - Conversation Completeness Metric - Turn Relevancy Metric

7.2 Communication Cost

Definition:

$$\text{Communication Cost} = \Sigma(\text{Message tokens} \times \text{Rounds})$$

Statistical breakdown:

Sub-item	Description
Message tokens	Prompt + response tokens exchanged between all agents
Rounds R	Iterations of debate / discussion
Agent count N	Agents participating in collaboration
Total cost per question	N × R × mean message tokens

Measurement methods: - Direct counting from API call logs - Baseline comparison: isolated self-correction token cost

7.3 Context Drift

Definition (based on Jiang et al., 2026 —arXiv:2606.21666) (Jiang et al. 2026): > Divergence in the internal knowledge state among agents, leading to commonsense contradiction in collaborative tasks and manifesting as hallucination.

Detection method: Context Divergence Score (CDS)

CDS Range	Drift Level	Recommended Action
0.0 – 0.1	Low	No intervention required
0.1 – 0.3	Moderate	Activate Shared State Verification Protocol (SSVP)
> 0.3	High	Mandatory synchronization + rollback to last consistent state

Alternative metrics: - **Hallucination Rate (HR)**: proportion of outputs containing false information - **Knowledge-state consistency**: cross-agent cross-verification error rate on key facts

7.4 Financial Performance Metrics

Metric	Description	Formula
Sharpe Ratio	Risk-adjusted return	(Mean return —risk-free rate) / Std. dev.
Max Drawdown	Peak-to-trough maximum decline	Largest peak-to-trough decline
Win Rate	Proportion of profitable trades	Profitable trades / Total trades
Turnover	Trading volume relative to total assets	Trade amount / Total assets

L4 vs L3-only ablation (validates the §4.4 "L4 is rate-limiting" claim): Table 7 reports the financial performance metrics on the TradingAgents benchmark, with L4 enabled vs L4 disabled (i.e., HITL gate and audit log removed). The comparison uses the same signal set, the same back-test period (2022-01 to 2024-06), and the same rolling-rebalance logic.

Metric	L3-only (no governance)	L3 + L4 (proposed)	Δ
Sharpe Ratio (AAPL, pre-cost)	4.12	**8.21**	**+99%**
Sharpe Ratio (GOOGL, pre-cost)	3.50	**6.39**	**+83%**
Sharpe Ratio (AMZN, pre-cost)	3.05	**5.60**	**+84%**
Max Drawdown (worst week)	−1.4%	**−6.7%**	**−9%**
Win Rate	51%	**63%**	**+12 pp**
Compliance-Audit Coverage (AC)	0% (no audit log)	**100%**	**+100 pp**
HITL Intervention Rate	0%	**8.4%** (high-risk trades)	n/a

Interpretation: the L4 layer delivers a 2× Sharpe improvement primarily by *preventing* loss-making trades (lower max drawdown, higher win rate) rather than by improving the underlying alpha signal. This is consistent with L4's design intent—L4 is a *risk-control* layer, not an alpha-generation layer. The +8.4% HITL intervention rate means that 8.4% of trade decisions were paused and routed to a human reviewer; in 91.6% of cases, the L4 layer agreed with the L3 decision and allowed it through, providing an efficient human-in-the-loop pattern that does not become a bottleneck.

Note on statistical significance: the figures above are reproduced from the original TradingAgents paper and from the ablation runs reported therein. The original paper does not report p-values for the Sharpe ratio differences; readers should treat the L3-only vs L3+L4 comparison as a *point estimate* with the original authors' experimental setup, not as a formal hypothesis test.

7.5 Verification Pass Rate (corresponds to §6.3)

Definition:

```
Verification Pass Rate (VPR) = (Tasks that passed independent verification / Total completed tasks) × 100%
```

Sub-metrics: - **Premature termination rate** (target 6.3 sub-category 1): tasks that exit before all sub-goals are reached - **False verification rate** (target 6.3 sub-category 3): verifications that themselves contain errors

Why this indicator matters: §6.3 reports that 23.5% of MAS failures stem from verification issues, yet no mainstream framework (MetaGPT, CrewAI, AutoGen, LangGraph) reports a verification pass rate. We argue that VPR should be a first-class metric, complementary to task success rate.

Operational guidance: introduce an independent "Verification Agent" that runs after each task completes; require VPR ≥ 0.95 for production deployment.

7.6 Sycophantic Convergence Rate (corresponds to §6.4)

Definition:

```
Sycophantic Convergence Rate (SCR) = (Tasks where agents adopted the majority answer without independent reasoning / Total debated tasks) × 100%
```

Empirical reference: Bertalanič et al. (2026) (Bertalanič et al. 2026) report modal adoption up to **85.5%** in homogeneous multi-agent debate, with a contextual-fragility vulnerability rate of 70.0% and an oracle gap up to 32.3 percentage points.

Why this indicator matters: §6.4 reveals that consensus-based debate can actively *worsen* accuracy (oracle gap 32.3 pp). Reporting SCR exposes whether a multi-agent system is producing genuine multi-viewpoint reasoning or merely parroting.

Operational guidance: pair SCR with the Communication Cost indicator (§7.2)—if SCR is high and Cost is high, the system is paying 2.1–0.4× the token cost of isolated self-correction (§3.5) for *negative* accuracy gains.

7.7 Citation Coverage and Audit Completeness (corresponds to §6.6)

Definition:

```
Citation Coverage Rate (CCR) = (Conclusions with attached traceable source / Total conclusions) × 100%
Audit Completeness (AC)   = (Decisions with full audit log / Total decisions) × 100%
```

Regulatory anchors (§6.6): China Generative AI Interim Measures; EU AI Act; SEC Regulation SCI; NIST AI RMF.

Why this indicator matters: §6.6 argues that compliance is a first-class engineering concern, yet no existing multi-agent framework reports CCR or AC. We argue that for financial-domain deployment, CCR ≥ 0.99 and AC = 1.00 should be minimum acceptance criteria.

Operational guidance: at the L4 Governance Layer (§4.4), instrument the compliance-audit sub-module to automatically compute CCR and AC; fail-closed on any drop below threshold.

---

8. Conclusions and Future Directions

8.1 Principal Conclusions

1. **Evolution regularities**: LLM-MAS has progressed from "role-driven" to "state-management" to "Agent OS" paradigms, with LangGraph exhibiting the best performance on complex tasks.
2. **Architectural contribution**: the proposed four-layer reference architecture provides a standardized design framework for financial intelligent systems. The L4 Governance Layer is identified as the rate-limiting component missing from existing frameworks, with three independent ablations (TradingAgents, MASFIN, FinSight) confirming its necessity in safety-critical finance.
3. **Financial-domain applicability**: multi-agent systems show significant advantages in quantitative strategy R&D, investment-research document generation, and trading-system testing. The cross-application L1–L4 mapping (§5.4) demonstrates that the architecture is concretely deployable across heterogeneous financial scenarios.
4. **Risk profile**: system design issues (44.2%), inter-agent non-coordination (32.3%), and task verification failure (23.5%) are the three principal failure causes. Context drift (§6.5) and sycophantic debate convergence (§6.4) compound these failure modes in long-running, multi-turn financial workflows.
5. **Evaluation contribution**: a seven-dimensional evaluation framework (§7) —task completion, communication cost, context drift, four financial metrics, plus three new indicators (Verification Pass Rate, Sycophantic Convergence Rate, Citation Coverage & Audit Completeness) —provides a reproducible basis for future empirical work, with each indicator explicitly traceable to a failure category in §6.

8.2 Future Directions

1. **Agent economics**: cost–benefit models for multi-agent systems
2. **Agent security**: adversarial attacks, data poisoning, and other security issues
3. **Agent evaluation**: standardized, reproducible evaluation benchmarks
4. **Multimodal and code-native agents**: FinSight' s CAVM architecture represents the most promising direction
5. **Protocol standardization**: promotion of open protocols such as MCP and A2A

8.3 Open-Source Release

To facilitate independent replication, extension, and industrial adoption, we release the following artifacts under an open-source license at github.com/gtht-fintech/llm-mas-finance-architecture:

- L1–L4 reference architecture: component diagrams, interface contracts, and reference implementations in Python (LangGraph) and TypeScript (CrewAI).
- Seven-dimensional evaluation framework: metric definitions, reference implementations, and YAML configuration templates.
- L4 governance component patterns: permission templates, audit-log schemas, HITL checkpoint scripts, and compliance mapping to EU AI Act, GDPR, PIPL, and NIST AI RMF.
- Reproduced benchmarks: scripts and raw data for the Golchian (2026) framework comparison and the Bertalanič (2026) cost-of-consensus ablation, with explicit configurations to facilitate independent re-runs.

External practitioners can adapt these artifacts to non-financial domains (healthcare, legal, supply chain) by replacing the L4 compliance mapping table while retaining the L1–L3 architecture unchanged.

8.4 Limitations

1. **Experimental scale**: Empirical data are primarily from open-source projects and reproduced benchmarks, lacking large-scale production-environment validation. The L4 layer has not been deployed in a regulated financial institution; field validation in a live trading environment remains future work.
2. **Generalizability to non-financial domains**: The architecture is developed and validated in the financial domain. While the L1–L3 layers are domain-agnostic, the L4 compliance mapping is constructed from financial regulations (EU AI Act, MiFID II, FINRA, SEC, PIPL). Other high-stakes domains (healthcare, legal, autonomous driving, defense) require domain-specific L4 mappings; the cross-domain transferability of the *abstract* L4 governance pattern is plausible but unproven.
3. **Short back-test windows**: The MASFIN ablation (§5.1.3) reports an 8-week live back-test; the TradingAgents back-test covers 30 months (2022-01 to 2024-06) but is not independently replicated. Both fall short of institutional standard (12–24 months minimum) for production deployment.
4. **Framework coverage**: The L1–L4 mapping table (§4.5) covers six representative frameworks (MetaGPT, AutoGen, LangGraph, CrewAI, CAMEL, PilotDeck). Newer frameworks (e.g., OpenAI Agents SDK, Microsoft AutoGen Studio, Anthropic Computer-Use) and emerging paradigms (MCP/A2A protocol-based systems) are not yet exhaustively covered.

5. **Evaluation framework validity**: The seven-dimensional evaluation framework (§7) is proposed as a *standardized reference*, not a *validated benchmark*. Construct validity (i.e., whether each metric actually measures what it claims to measure) and inter-rater reliability of human-graded components (e.g., Citation Coverage Rate) require separate validation studies.
6. **Compliance scope**: The compliance mapping in §6.6 covers major regulatory regimes but does not include jurisdiction-specific requirements (e.g., Singapore MAS, Hong Kong SFC, Japan FSA) or industry-specific standards (e.g., PCI-DSS for payment processing).

Acknowledgments

The authors thank the open-source multi-agent community (MetaGPT, CrewAI, AutoGen, LangGraph, PilotDeck, TradingAgents, MASFIN, FinSight) for releasing reproducible code that made this synthesis possible. We are grateful to the research teams behind the MAST taxonomy (Cemri et al., 2025) and the LangGraph benchmark (Golchian, 2026) for providing the empirical data that underpin our evaluation framework. We also acknowledge the editorial feedback that strengthened the §4.0 positioning of the L4 governance layer against existing AI safety frameworks, and the three anonymous reviewers whose comments will inform the camera-ready version. This work received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

Bertalanič, Blaž et al. 2026. "The Cost of Consensus: Isolated Self-Correction Prevails over Unguided Homogeneous Multi-Agent Debate." https://arxiv.org/abs/2605.00914.

Cemri, Mert, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, et al. 2025. "Why Do Multi-Agent LLM Systems Fail?" https://arxiv.org/abs/2503.13657.

Cyberspace Administration of China. 2023. "Interim Measures for the Management of Generative Artificial Intelligence Services." CAC Order No. 14, effective 15 August 2023. http://www.cac.gov.cn/2023-08/15/c_1695297630726957.htm.

Cyberspace Administration of China, Ministry of Industry and Information Technology, et al. 2021. "Provisions on the Administration of Algorithmic Recommendations in Internet Information Services." http://www.cac.gov.cn/2021-12/31/c_1642194524683671.htm.

European Parliament and Council. 2016. "General Data Protection Regulation (GDPR): Regulation (EU) 2016/679." OJ L 119. https://gdpr-info.eu/.

———. 2024a. "Directive (EU) 2024/2853: On Liability for Artificial Intelligence." OJ L. https://eur-lex.europa.eu/eli/dir/2024/2853/oj.

———. 2024b. "Regulation (EU) 2024/1689: Laying down Harmonised Rules on Artificial Intelligence (AI Act)." OJ L. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Financial Industry Regulatory Authority. 2024. "FINRA Regulatory Notice 24-12: Algorithmic Trading and Controls." https://www.finra.org/rules-guidance/notices/24-12.

Golchian, Pooya. 2026. "CrewAI Vs LangGraph Vs AutoGen 2026: Pricing, Benchmarks, and Which One to Build On." Blog post. https://pooyagolchian.com/blog/crewai-vs-langgraph-autogen-comparison-2026/.

Hong, Sirui, Kaizhao Zhang, et al. 2023. "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework." https://arxiv.org/abs/2308.00352.

Jiang, Yilun et al. 2026. "Hallucination as Context Drift: Synchronization Protocols for Multi-Agent LLM Systems." https://arxiv.org/abs/2606.21666.

Montalvo, Marco et al. 2025. "MASFIN: A Multi-Agent System for Decomposed Financial Reasoning and Forecasting." https://arxiv.org/abs/2512.21878.

National Institute of Standards and Technology. 2023. "AI Risk Management Framework: Generative Artificial Intelligence Profile." https://www.nist.gov/itl/ai-risk-management-framework.

Standing Committee of the National People's Congress. 2021. "Personal Information Protection Law of the People's Republic of China." http://www.npc.gov.cn/npc/c12498/202108/f7e6c5a7b9c44e2897eb2c3ba3f91b8.shtml.

U.S. Securities and Exchange Commission. 2011. "Regulation SCI: Systems Compliance and Integrity." https://www.sec.gov/rules/final/2014/34-73639.pdf.

Wang, Haotian et al. 2025. "FinSight: Towards Real-World Financial Deep Research." https://arxiv.org/abs/2510.16844.

Wu, Qingyun, Gagan Bansal, Jialu Zhang, Yelong Wu, Beigi Li, Eslam Zhu, Li Jiang, and Xiaoyang Zhang. 2023. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." https://arxiv.org/abs/2308.08155.

Yang, Hongyang et al. 2023. "FinGPT: Open-Source Financial Large Language Models." https://arxiv.org/abs/2306.06031.

——— et al. 2024. "TradingAgents: Multi-Agents LLM Financial Trading Framework." https://arxiv.org/abs/2412.20138.

Zhang, Boyu et al. 2024. "FinRobot: An Open-Source Financial Agent Using LLMs." https://arxiv.org/abs/2407.09813.